

scitech-librarian — Benutzerhandbuch

Version 3.5.0

2026-09-04

Inhaltsverzeichnis

1. Was es ist	2
2. Installation und Einrichtung	2
3. Konzepte	3
4. Abfragen schreiben	3
5. Eine Suche ausführen	4
6. Das Forschungsverzeichnis	5
6.1 Index	5
6.2 Datensätze von außen einbringen	5
6.3 Aus Zotero, Mendeley und EndNote	6
7. Berichte	6
7.1 Ein Lauf	6
7.2 Das ganze Forschungsverzeichnis	6
7.3 Stufen	6
7.4 Formate	7
7.5 Filter	7
7.6 PRISMA	7
7.7 Vorschläge	8
7.8 Sprachen	8
8. Zeitschriftenkennzahlen	8
9. Web of Science	9
10. Logs und Audit	9
11. Arbeitsabläufe	9
12. Funktionen und Einschränkungen	10
13. Tests	10
14. Lizenz und Verhalten	11

[English](#) · [Português \(Brasil\)](#) · [Español](#) · **Deutsch** · [Français](#)

Übersetzung des englischen Handbuchs, das maßgeblich bleibt; Befehle, Dateinamen, Optionen und Codeblöcke sind unverändert übernommen.

1. Was es ist

scitech-librarian ist ein reproduzierbares Literaturrecherche-Instrument für Naturwissenschaft und Technik. Sie schreiben eine strukturierte Abfrage einmal; es führt sie über deren dokumentierte APIs gegen bis zu neun bibliografische Datenbanken aus, archiviert alles (Datensätze, den exakten an jede Datenbank gesendeten Abfragestring, Trefferzahlen, ein Log) und schreibt einen Literaturrecherche-Bericht mit PRISMA-2020-Flussdiagramm. Über Monate sammeln sich die Läufe, plus auf anderen Wegen beschaffte Datensätze, in einem **Forschungsverzeichnis**, das derselbe Bericht als Ganzes beschreiben kann — was jede Suche beigetragen hat, was jede Datenbank beigesteuert hat, wie die Trefferzahlen gedriftet sind, welche Zeitschriften zählen.

Es besteht aus fünf Python-Skripten plus zwei gemeinsamen Modulen (`render.py`, `i18n.py`), ohne Abhängigkeiten jenseits der Standardbibliothek. Es gibt nichts zu installieren: Dateien kopieren, Schlüssel bereitstellen, `queries.json` schreiben, ausführen.

Datei	Rolle
<code>librarian.py</code>	eine Suche ausführen; einen Lauf archivieren; den Bericht aufrufen
<code>project.py</code>	Forschungsverzeichnis: Index, Import externer Datensätze, Status
<code>report.py</code>	Berichte für einen Lauf oder das ganze Verzeichnis; PRISMA; Filter
<code>journals.py</code>	Zeitschriftenkennzahlen (Impact-Factor-ähnliche Werte) pro Jahr
<code>wos_manual.py</code>	Web of Science von Hand (keine brauchbare kostenlose API)
<code>render.py</code>	Markdown- / HTML- / LaTeX- / Text-Renderer und die PDF-Kette (von <code>report.py</code> importiert)
<code>i18n.py</code>	Berichtssprachen: der en / pt-BR / es / de / fr-Katalog (von <code>report.py</code> importiert; §7.8)

Für KI-Agenten. `AGENTS.md` im Wurzelverzeichnis des Repositorys ist die vollständige maschinenorientierte Beschreibung des Werkzeugs. Wenn Sie mit einem Coding-Agenten arbeiten (Claude Code, Codex, Cursor...), sagen Sie ihm: „*Lies AGENTS.md und führe dann eine Neuheitsprüfung zu X durch*“ — sie enthält die Befehle, Dateischemata, Arbeitsabläufe und die Regeln, die der Agent nicht brechen darf.

2. Installation und Einrichtung

Voraussetzungen: Python 3.9 oder neuer. Optional, für gesetzte PDF-Berichte: eine LaTeX-Distribution (xelatex, lualatex oder pdflatex) oder pandoc; ohne sie erzeugt ein eingebauter Klartext-Writer das PDF.

```
git clone https://github.com/fabiocampolim-design/scitech-librarian
cd scitech-librarian
cp .env.example .env           # fill in what you have (or use the environment)
cp queries.example.json queries.json
python librarian.py --selftest
```

Die Schlüssel werden aus der Prozessumgebung gelesen. Kopieren Sie `.env.example` nach `.env` und füllen Sie diese Datei aus, oder setzen Sie dieselben Variablennamen in Ihrer Shell, in der Konfiguration Ihres Agenten oder Starters, oder als CI-Secrets — `.env` liefert nur, was die Umgebung noch nicht gesetzt hat; eine außerhalb gesetzte Variable gewinnt also immer und die Datei ist optional.

Schlüssel:

Schlüssel	Nötig für	Beschaffung
CONTACT_EMAIL	„polite pool“-Zugang zu OpenAlex/Crossref/Unpaywall	Ihre Adresse
ADS_TOKEN	NASA ADS	kostenlos, https://ui.adsabs.harvard.edu/user/settings/token
SCOPUS_API_KEY	Scopus (+ Institutionsnetz/VPN)	kostenlos, https://dev.elsevier.com/apikey/manage
SCOPUS_INSTTOKEN	Scopus ohne VPN	bei Ihrer Bibliothek erfragen
S2_API_KEY	schnelleres Semantic Scholar	optional
CORE_API_KEY	CORE — optional; anonyme Aufrufe funktionieren, unterliegen aber einem Ratenlimit	kostenlos, https://core.ac.uk/services/api
WOS_STARTER_KEY	Web of Science Starter API (eingeschränkte Grammatik)	selten lohnend

Sechs Backends (OpenAlex, arXiv, INSPIRE-HEP, Semantic Scholar, Crossref, CORE) brauchen weder Schlüssel noch Institution. `python librarian.py --list` meldet, welche Schlüssel auf einem der beiden Wege gefunden wurden; `--selftest` beweist, dass sie funktionieren.

Einbettung in ein anderes Projekt. Legen Sie die sieben Dateien in ein Unterverzeichnis `tools/`; `.env`, `queries.json` und `lit/` werden dann im übergeordneten Verzeichnis gesucht.

3. Konzepte

Block. Eine strukturierte Abfrage: eine Liste von Synonymgruppen, mit AND verknüpft, jede Gruppe eine Liste von Synonymen, mit OR verknüpft. Ein Block hat einen Namen (`A`, `CD`, `NOV`...), einen Titel und eine Notiz. Blöcke leben in `queries.json`.

Lauf. Eine Ausführung von `librarian.py`: jeder gewählte Block gegen jedes gewählte Backend, archiviert unter `lit/runs/<timestamp>/`.

Forschungsverzeichnis. Ein Ordner (Standard `lit/`, ein anderer mit `--outdir`), der alle Läufe eines Projekts enthält, die von außen importierten Datensätze, den Projektindex (`project.json`), die PRISMA-Sichtungszahlen (`screening.json`), Zeitschriftenkennzahlen, Berichte und Audit-Logs. Ein Verzeichnis pro Projekt; ein Labor hat mehrere.

Manuelle Quelle. Datensätze, die nicht aus einem Lauf stammen: ein Zotero- oder Mendeley-Export, die RIS-Datei einer Kollegin, eine Web-of-Science-Sitzung, eine Literaturliste. Mit `project.py ingest` importiert, behalten sie ihre Herkunft (wer, wann, woher, Methode) und erscheinen in jedem Bericht als eine weitere Quelle, und im PRISMA-Fluss in der rechten Spalte.

Datensatz. Das gemeinsame Schema, das jede Datei verwendet: `title year doi journal authors url abstract cited_by issn block backend`. Zusammengeführte Projektdatensätze tragen zusätzlich `found_by` (welche Quellen ihn fanden) und `first_seen`.

Stufe. Wie viel ein Bericht enthält: `simple` (wenige Seiten), `intermediate` (jeder eindeutige Datensatz plus Auswertungen), `full` (alles, Abstracts inklusive — Hunderte Seiten bei großen Projekten).

4. Abfragen schreiben

`queries.json`:

```
{
  "PSC": {
    "title": "Perovskite solar cell degradation under humidity",
    "note": "grounding block -- expect thousands",
```

```

"groups": [["perovskite solar cell", "halide perovskite photovoltaic"],
           ["degradation", "stability"],
           ["humidity", "moisture"]]
},
"NOV": {
  "title": "novelty check: origami metamaterials as topological acoustic pumps",
  "note": "a SMALL number is the good outcome; read every hit",
  "groups": [["origami", "kirigami"], ["acoustic metamaterial", "phononic crystal"],
             ["topological pumping", "edge state"]],
  "arxiv_groups": [0, 2]
}
}

```

Faustregeln:

- Setzen Sie Begriffe nicht in Anführungszeichen; das Werkzeug tut das für die Grammatik jeder Datenbank.
- Ein einzelnes generisches Wort (`model`, `structure`, `system`) in einer eigenen Gruppe ist die übliche Ursache für Trefferzahlen in den Zehntausenden.
- `arxiv_groups` benennt, welche (höchstens zwei) Gruppen arXiv erhält; arXiv hängt bei tief verschachtelten Booleans. Standard: die ersten beiden. arXiv wird in Seiten zu 100 Datensätzen mit 3 s Pause abgerufen, ein großes `--limit` ist dort also langsam.
- Der informativste Block ist eine bewusste Schnittmenge zweier Literaturen, von denen Sie vermuten, dass sie nicht miteinander reden. Ein Ergebnis nahe null ist ein Befund — *wenn* Sie danach jeden Treffer lesen.
- Nachbarschaftsoperatoren (`NEAR/n`, `W/n`) lassen sich nicht ausdrücken; wenn Ihr Aufsatz sie braucht, führen Sie handgeschriebene Web-of-Science- / Scopus-Strings daneben und zitieren diese.

5. Eine Suche ausführen

```

python librarian.py                # all blocks, every configured backend
python librarian.py --counts-only  # hit counts only (seconds)
python librarian.py --blocks NOV CD # selected blocks
python librarian.py --backends openalex ads
python librarian.py --skip arxiv   # drop a misbehaving backend
python librarian.py --limit 500    # records per block and backend (default 300)
python librarian.py --pdfs --pdf-blocks NOV # legal OA-PDF links via Unpaywall
python librarian.py --keep-junk    # keep non-curated venues (Zenodo, SSRN...)
python librarian.py --outdir lit_topomat # another research directory
python librarian.py --report-level intermediate --report-format md html pdf
python librarian.py --report-lang pt-BR # report in Portuguese (en, pt-BR, es, de, fr; §7.8)
python librarian.py --no-report
python librarian.py --queries other.json # another query file (default ./queries.json)
python librarian.py --backends-file b.json # another backends config; --init-backends writes the default
python librarian.py --timeout 60          # per-request timeout in seconds (default 45)
python librarian.py --list                # blocks and backend readiness, then exit
python librarian.py --selftest            # ping every backend, then exit

```

Vollständige Parameterliste: `python librarian.py --help`. Jede Option hat eine Voreinstellung; `--outdir`, `--verbose`, `--quiet` und `--log-dir` gibt es bei jedem Skript.

Was ein Lauf schreibt (`lit/runs/<stamp>/`):

Datei	Inhalt
<code>counts.json</code> , <code>counts.md</code>	Trefferzahlen pro Block und Backend; eingefüfertige Tabelle
<code>queries.json</code>	der exakte an jedes Backend gesendete Abfragestring
<code>blocks.json</code>	die verwendeten Blockdefinitionen

Datei	Inhalt
meta.json	Einstellungen, Backends und Endpunkte, Version, Zeiten
records/<block>_<backend>.json	Rohdatensätze pro Backend (nach dem Zeitschriftenfilter)
ris/<block>_<backend>.ris	RIS pro Block für Zotero/Mendeley/EndNote
all_records.json/.csv/.ris	dedupliziert, nach Zitationen sortiert
all_records.bib, all_records.csl.json	derselbe Bestand als BibTeX und CSL-JSON
junk.json	vom Zeitschriftenfilter entfernte Datensätze, mit ihren Zeitschriften
prisma.json	Vorlage für die manuellen PRISMA-Stufen
run.log	alles, was ausgegeben wurde
report.*	der Bericht (siehe §7)

Plus `lit/counts_history.csv` (eine Zeile pro Block/Backend/Lauf, für die Drift) und `lit/logs/librarian_<stamp>_<pid>.log` (Audit-Log: Aufruf, Versionen, jede Meldung).

Trefferzahlen werden nach jedem API-Aufruf per Checkpoint gesichert, und Strg-C ist sicher: ein Hänger spät in einem langen Lauf verliert nichts.

6. Das Forschungsverzeichnis

6.1 Index

```
python project.py init --name "Topological materials review" --description "..."  
python project.py status
```

`status` listet jedes Mitglied (Läufe und manuelle Quellen) mit Datum, Datensatzzahl, Methode und Label, den Zustand des Eingangsordners und den letzten Bericht. Mitglieder werden durch Auflisten des Verzeichnisses entdeckt — nichts muss deklariert werden. `project.json` enthält nur, was sich nicht entdecken lässt:

```
{"name": "...", "description": "...", "created": "2026-08-28",  
  "exclude": ["20260814T223331"],  
  "labels": {"20260828T095041": "August full scan"},  
  "block_aliases": {"X": "CD"},  
  "defaults": {"level": "simple", "format": ["md"], "metric": "openalex_2yr"}}
```

```
python project.py exclude 20260814T223331      # a test run you do not want in reports  
python project.py label 20260828T095041 "August full scan"  
python project.py alias X CD                  # block renamed between runs  
python project.py oa                          # Unpaywall pass over every member that lacks OA data  
python project.py oa --members 20260828T095041 # restrict the pass to these member ids
```

`oa` ist die nachträgliche Open-Access-Abfrage: Läufe ohne `--pdfs` und manuelle Quellen erhalten die Felder `is_oa` / `oa_pdf` (nur legale Kopien, zwischengespeichert in `unpaywall_cache.json`), die die OA-Statistik des Berichts und `--oa-only` dann für das ganze Projekt abdecken.

6.2 Datensätze von außen einbringen

Drei Wege, alle enden in `lit/manual/<name>/` mit der Originaldatei, einer `records.json` im gemeinsamen Schema und einer `source.json` mit der Herkunft:

1. **Kommandozeile** — der vollständig beschriebene Weg:

```
python project.py ingest export.ris --name zotero-aug --block CD \  
  --method citation --who "A. Colleague" --origin "Zotero group library" \  
  --note "reference lists of the three key papers"
```

Mehrere Dateien können angegeben werden; `--kind` überschreibt die Erkennung per Endung (`ris`, `bibtex`, `csv`, `json`).

2. **Eingangsortner** — Dateien in `lit/inbox/` ablegen und `python project.py ingest --inbox` ausführen; jede Datei wird eine nach ihr benannte Quelle (`--method` usw. hinzufügen, um es auf alle anzuwenden).
3. **Web of Science** — `python wos_manual.py ingest` liest die RIS-Dateien, die Sie aus der WoS-Oberfläche exportiert haben, und registriert sie als manuelle Quellen mit `method=database`.

Akzeptierte Formate: RIS (Zotero, Mendeley, EndNote, Web of Science, Scopus), BibTeX, CSV mit Kopfzeile (Scopus- und WoS-Spaltennamen werden erkannt; sonst `title`, `year`, `doi`, `journal`, `authors`, `url`, `abstract`, `block`, `cited_by`) und JSON-Datensatzlisten (etwa `all_records.json` aus dem Lauf einer Kollegin).

`--method` folgt den PRISMA-2020-Kategorien für über andere Methoden identifizierte Datensätze: `database` (ein Datenbankexport — geht in die Datenbankspalte), `citation` (Literaturlisten, zitierende Arbeiten), `website`, `organisation`, `expert` (die Empfehlung einer Kollegin), `other`.

Sie können einem einzelnen Bericht auch zusätzliche Dateien mitgeben, ohne sie zu speichern: `report.py --records file.ris`.

6.3 Aus Zotero, Mendeley und EndNote

Hinaus: jeder Lauf schreibt RIS (`all_records.ris`, `ris/` pro Block), BibTeX (`all_records.bib`) und CSL-JSON (`all_records.csl.json`); importieren mit File → Import. Abstracts, DOIs und URLs werden mitgeführt, und der Blockname kommt als Schlagwort (`block:NOV`) an, sodass die importierten Einträge vorab getaggt sind.

Hinein: eine Sammlung als RIS exportieren (Zotero: Rechtsklick → Export Collection → RIS; Mendeley: File → Export → RIS; EndNoteX: File → Export → RefMan RIS) und wie oben importieren. Es gibt keine Live-Verbindung zur Zotero-API (Fahrplan).

7. Berichte

7.1 Ein Lauf

```
python report.py lit/runs/20260828T095041
python report.py --latest --level full --format html pdf
```

7.2 Das ganze Forschungsverzeichnis

```
python report.py --project
python report.py --project --outdir lit_topomat --level intermediate --format md html
```

Berichte gehen nach `lit/reports/<stamp>-<level>/`. Der Projektbericht ergänzt eine **Quellen**-Tabelle (jeder Lauf und jede manuelle Quelle, ihr Datum, Methode, Datensätze und „neu hier“ — die eindeutigen Datensätze, die keine frühere Quelle gefunden hatte), einen **Zeitverlauf** (Trefferzahlen pro Block über die Läufe; wann Datensätze ins Projekt kamen), einen PRISMA-Fluss mit beiden Identifikationsspalten und, wenn `journals.py` gelaufen ist, Zeitschriftenkennzahlen.

7.3 Stufen

Stufe	Abschnitte
simple	Metadaten; Quellen; Suchstrategie mit dem exakten String pro Backend; Ergebniszusammenfassung; Zeitverlauf; PRISMA-2020-Fluss + PRISMA-S-Checkliste; Top-10-Datensätze pro Block; Vorschläge

Stufe	Abschnitte
intermediate	+ jeder eindeutige Datensatz; Quellenüberschneidung („nur hier gefunden“); Verteilungen nach Jahr / Zeitschrift / Autor; Zeitschriftenkennzahlen; gefilterte Zeitschriften; Fehler; Open-Access-Statistik; Trefferverlauf
full	+ jeder Datensatz mit vollständigem Abstract, Autorenliste und den Quellen, die ihn fanden; Rohlisten pro Quelle vor der Deduplizierung; die gefilterten Datensätze; Backend-Konfiguration; project.json- und source.json-Dateien; das Lauf-Log; Umgebung

Umfang, nach dem mitgelieferten Beispiel (vier Blöcke, drei CC0-Datenbanken, 1.226 eindeutige Datensätze): 6, 68 und 427 PDF-Seiten.

7.4 Formate

md (Markdown; rendert auf GitHub), **html** (eigenständig, hell/dunkel, druckbar, SVG-Diagramm), **tex** (LaTeX mit TikZ-Diagramm), **pdf**, **txt** (reiner Text, ASCII-Diagramm). Das PDF wird aus dem LaTeX mit xelatex, lualatex oder pdflatex kompiliert, wenn eines installiert ist, sonst mit pandoc, sonst von einem eingebauten Writer, der die Textversion setzt — die Option schlägt nie fehl.

7.5 Filter

Option	Wirkung
<code>--since DATE, --until DATE</code>	Mitglieder (Läufe / manuelle Quellen) behalten, die im Fenster gesucht wurden
<code>--latest</code>	nur das jüngste Mitglied (Projekt); der neueste Lauf (Einzelmodus)
<code>--diff</code>	nur Datensätze behalten, die <i>erstmal</i> s im Fenster gesehen wurden — „was die Suchen seit DATE beigetragen haben“
<code>--year-from Y, --year-to Y</code>	Erscheinungsjahr
<code>--backends a b</code>	einzubeziehende Datenbanken / Quellen (manuelle Quellen sind <code>manual:<name></code>)
<code>--blocks A CD</code>	einzubeziehende Blöcke
<code>--sources auto\ manual\ all</code>	Mitgliedsarten
<code>--records FILE...</code>	zusätzliche RIS/BibTeX/CSV/JSON als vorübergehende manuelle Quelle
<code>--metric NAME --min-metric X</code>	Datensätze behalten, deren Zeitschriftenkennzahl mindestens X ist (siehe §8)
<code>--min-citations N</code>	Zitationsschwelle
<code>--oa-only</code>	nur Datensätze mit legaler Open-Access-Kopie (braucht Daten von <code>--pdfs</code> oder <code>project.py oa</code>)
<code>--top N, --sort cited\ year\ metric</code>	Tabellengröße und -reihenfolge
<code>--basename, --out</code>	Dateinamenstamm und Ausgabeverzeichnis

Filter werden in der Metadatentabelle des Berichts und in PRISMA-S-Punkt 9 aufgeführt, sodass ein gefilterter Bericht nie mit der ganzen Suche verwechselt wird.

7.6 PRISMA

Der Bericht enthält ein PRISMA-2020-Flussdiagramm (SVG in HTML, TikZ in LaTeX/PDF, ASCII in Markdown und Text) und eine PRISMA-S-Checkliste zur Suchberichterstattung. Das Werkzeug füllt aus, was es wissen kann:

identifizierte Datensätze pro Datenbank (im Projektmodus über die Läufe summiert), über andere Methoden identifizierte Datensätze (manuelle Quellen nach Methode), abgerufene Datensätze, durch Automatisierung entfernte (der Zeitschriftenfilter), entfernte Duplikate, verbleibende zu sichtende Datensätze. Es ist ausdrücklich, dass *identifiziert* (was jede Datenbank meldet) und *abgerufen* (was innerhalb von `--limit` heruntergeladen wurde) verschieden sind.

Die Stufen, die nur ein Mensch kennen kann, werden aus `prisma.json` (Einzellauf) oder `screening.json` (Forschungsverzeichnis) gelesen; beim ersten Bericht wird eine Vorlage mit `null`-Werten geschrieben. Tragen Sie die Ganzzahlen ein, während die Sichtung voranschreitet, und führen Sie den Bericht erneut aus:

```
{"records_screened": 2216, "records_excluded": 2100,
 "reports_sought": 116, "reports_not_retrieved": 4, "reports_assessed": 112,
 "excluded_reasons": {"not_topological": 60, "no_experiment": 30},
 "other_sought": 12, "other_not_retrieved": 0, "other_assessed": 12,
 "other_excluded_reasons": {"duplicate_of_included": 3},
 "studies_included": 22, "reports_included": 31,
 "citation_searching": "reference lists of the 22 included studies",
 "prior_work": "none", "peer_review": "search strategy reviewed by the librarian"}
```

7.7 Vorschläge

Regelbasiert, am Ende jedes Berichts: fehlgeschlagene Backend-Aufrufe, Blöcke mit Tausenden Treffern, Blöcke in Neuheitsgröße (jeden Treffer lesen), erreichte `--limit`-Obergrenzen, eine Datenbank mit hohem Anteil gefilterter Zeitschriften, kein zitierfähiges Backend, Open-Access-Abfrage nicht ausgeführt, unausgefüllte PRISMA-Stufen, keine Zeitschriftenkennzahlen, Trefferdrift zwischen Läufen und — im Projektmodus — das Fehlen jeder manuellen Quelle.

7.8 Sprachen

```
python report.py --latest --lang pt-BR
python report.py --project --lang de --format pdf
python librarian.py --report-lang fr # the report written at the end of a run
```

`--lang` (`report.py`) und `--report-lang` (`librarian.py`) nehmen `en` (Standard), `pt-BR`, `es`, `de` oder `fr`; ein Forschungsverzeichnis kann seine eigene Voreinstellung mit `"defaults": {"lang": "es"}` in `project.json` festlegen, und eine explizite Option gewinnt darüber. Nur der berichtseigene Text ändert sich — Überschriften, Tabellenköpfe, die PRISMA-2020-Stufen und das Flussdiagramm in jedem Format, die PRISMA-S-Checkliste, die erläuternden Absätze und die Vorschläge — zusammen mit dem Tausendertrennzeichen der Sprache. Was das Werkzeug gefunden oder erhalten hat, wird in jeder Sprache exakt wiedergegeben: Titel, Abstracts, Autoren und Zeitschriften der Datensätze, Ihre Blocknamen und -notizen, die exakten Abfragestrings, Backend-Namen, die im Text zitierten Optionen, Dateinamen, die JSON-Ausgaben und das eingebettete Lauf-Log. Konsolenausgabe, `run.log` und die Audit-Logs sind immer englisch, sodass Läufe in verschiedenen Sprachen gemeinsam durchsuchbar bleiben.

8. Zeitschriftenkennzahlen

```
python journals.py fetch # every journal seen in the directory
python journals.py fetch --providers openalex --refresh
python journals.py import-scimago scimagojr_2024.csv --year 2024 [--all]
python journals.py import-jcr JCR_JournalResults_*.csv # Journal Citation Reports downloads
python journals.py import-csv other.csv --provider my_metric --year 2023 --name-col Journal --
value-col Value [--issn-col ISSN] [--delimiter ";"] # any name/value table; ISSN
python journals.py list --missing jcr_if # journals still to look up by hand
python journals.py show --metric scopus_citescore
```

Speicher: `lit/journals/metrics.json`, ein Eintrag pro Zeitschrift, nach ISSN (sonst normalisiertem Namen) indiziert, Werte **pro Jahr und nie überschrieben** — nächstes Jahr erneut abrufen, und der Bericht zeigt die

Reihe.

Anbieter	Schlüssel	Kennzahlen	Historie
openalex	keiner	openalex_2yr (mittlere 2-Jahres-Zitiertheit, ein Impact-Factor-ähnlicher Wert), openalex_h, Werke/Zitationen pro Jahr	Momentaufnahme unter dem Abrufjahr
scopus	SCOPUS_API_KEY	scopus_citescore, sjr, snip	volle Historie pro Jahr
scimago	keiner; die CSV des Jahres von scimagojr.com herunterladen	sjr, scimago_h, Quartil	eine Datei pro Jahr
jcr	Lizenz	jcr_if	nur Import

Der Journal Impact Factor (Clarivate JCR) ist proprietär: es gibt keine kostenlose API, und das Werkzeug wird ihn nicht scrapen. Lizenzierte Nutzer laden CSVs von der JCR-Seite *Browse journals* herunter (600 Zeilen pro Download; nach Kategorie, dann nach Quartil aufteilen) und importieren sie mit `journals.py import-jcr FILE...` — Spalten und JIF-Jahr werden erkannt. `journals.py list --missing jcr_if` gibt die Zeitschriften Ihres Verzeichnisses ohne Wert aus, also die nachzuschlagende Liste. Das vollständige Protokoll steht in `docs/JCR_IMPORT.md`. Für eine Kennzahl, die jede Zeitschrift abdeckt, ist die SCImago-CSV (~30.000 Zeitschriften, ein Download) der praktische Weg; `--all` importiert die ganze Datei, die Voreinstellung nur die in Ihren Datensätzen gesehenen Zeitschriften.

In Berichten: eine Kennzahlspalte in Datensatztabellen, „Zeitschriften in diesem Bestand nach Kennzahl“, eine Entwicklungstabelle für Zeitschriften mit zwei oder mehr erfassten Jahren und der Filter `--min-metric`. `--metric` wählt welche (Standard `openalex_2yr` oder `defaults.metric` in `project.json`).

9. Web of Science

Die vollständige TS=/NEAR-Grammatik liegt in der selten lizenzierten Expanded API; die kostenlose Starter-Stufe weist komplexe Booleans zurück. Web of Science ist daher Handarbeit, klein gemacht:

```
python wos_manual.py prep      # query files + CHECKLIST.md in WoS grammar
python wos_manual.py walk      # copies each query to the clipboard in turn
python wos_manual.py ingest     # RIS exports -> records, registered as manual sources
python wos_manual.py status
python wos_manual.py prep --queries other.json # a different query file (default ./queries.json)
```

Die Checkliste kodiert die Oberflächeneinstellungen, die Abfragen stillschweigend kaputtmachen (Core Collection, Advanced-Suche, Editionen, getaggte gegenüber nackter Form).

10. Logs und Audit

Jedes Skript schreibt `<outdir>/logs/<script>_<stamp>_<pid>.log` mit dem exakten Aufruf, Werkzeug- und Python-Versionen, dem Forschungsverzeichnis, jeder Warnung und jedem Fehler sowie dem Ergebnis. Die Konsolenausgabe ist standardmäßig knapp; `--verbose` zeigt alles, `--quiet` nur Warnungen und Fehler; `--log-dir` verlegt die Logs. Läufe bewahren zusätzlich `run.log` (die Konsolenmitschrift) im Laufverzeichnis.

11. Arbeitsabläufe

Eine Neuheitsprüfung (ein Nachmittag). 1–3 Kreuzabfrage-Blöcke schreiben; `--counts-only`; alles in den Tausenden verschärfen; vollständiger Lauf mit `--pdfs`; die Vorschläge lesen; jeden Treffer der kleinen Blöcke von Hand lesen; das Gesichtete in `prisma.json` festhalten; `report.py` erneut ausführen; RIS in Zotero importieren.

Eine systematische Suche über ein Projekt (Monate). `project.py` `init`. `librarian.py` in Abständen mit derselben `queries.json` erneut ausführen. Die Web-of-Science-Sitzungen und die Exporte der Kolleginnen importieren. `report.py --project` für das Gesamtbild; `--project --since <letzter Bericht> --diff` für das Neue; `journals.py fetch` jährlich. `screening.json` laufend ausfüllen; das PRISMA-Diagramm vervollständigt sich selbst, bereit für das Supplement.

Ein Labor. Ein Forschungsverzeichnis pro Projekt (`--outdir`); jedes hat eigenen Index, eigene Sichtung und Berichte. Eingangsordner lassen Mitarbeitende Exporte ablegen, ohne das Werkzeug zu lernen. Bewusst gibt es keine projektübergreifende Zusammenführung; andere Fragen, andere Blöcke.

Ein durchgerechnetes Beispiel. `docs/WALKTHROUGH.md` (englisch) führt ein reales Projekt von `queries.json` bis zum fertigen PRISMA-Diagramm, mit jedem Befehl.

Mit einem KI-Agenten. Verweisen Sie ihn auf `AGENTS.md`; bitten Sie ihn, `queries.json` aus Ihrer Forschungsfrage zu entwerfen, die Durchläufe zu starten und den Bericht mit Ihnen durchzugehen. Die strukturierte Abfragedatei, die JSON-Archive und der Bericht wurden dafür entworfen, von einem Agenten geschrieben und geprüft zu werden.

12. Funktionen und Einschränkungen

Funktionen: eine strukturelle Abfrage, in neun native Grammatiken übertragen; Datenbanken als JSON-Konfiguration (`--init-backends`); archivierte, zitierbare Läufe mit exakten Abfragestrings und Trefferverlauf; Checkpoints und sicheres Strg-C; ein Zeitschriftenfilter mit Belegen; sechs schlüssellose Backends; NASA ADS und INSPIRE für Physik; legale OA-PDF-Links über Unpaywall; dreistufige Berichte in fünf Formaten mit PRISMA 2020 und PRISMA-S; Forschungsverzeichnisse mit manuellen Quellen, Herkunft, Zeitverlauf und Differenzberichten; Zeitschriftenkennzahlen mit Jahresreihe; Audit-Logs; eine Offline-Testsuite (313 Prüfungen) und CI.

Einschränkungen, alle konstruktionsbedingt oder durch die Welt:

- Trefferzahlen sind zwischen Datenbanken nicht vergleichbar; Nachbarschaftsoperatoren werden verworfen. Hier entdecken; eine Datenbank im Aufsatz zitieren.
- Scopus-Ergebnisse erfordern institutionelle Berechtigung (Netz/VPN). Die API von Web of Science ist selten lizenziert; nutzen Sie den manuellen Weg.
- arXiv erhält höchstens zwei Gruppen pro Block.
- `--limit` begrenzt Datensätze pro Block und Backend (meistzitierte zuerst); große Blöcke sind ein Ausschnitt, nicht der vollständige Bestand. Erhöhen Sie es, wenn Sie Vollständigkeit brauchen.
- OpenAlex indexiert nicht kuratierte Repositorien (~15 % seiner Datensätze); standardmäßig gefiltert, in `junk.json` aufbewahrt.
- Kein PDF-Download (nur Unpaywall-Links), keine Schneeballsuche, kein Zitationsgraph, keine Live-Verbindung zu Zotero/Mendeley (Fahrplan); BibTeX und CSL-JSON werden geschrieben, nicht aus einer Zotero-Bibliothek zurückgelesen.
- Zeitschriftenkennzahlen: OpenAlex-Werte sind Momentaufnahmen; der JCR-Impact-Factor ist proprietär und nur importierbar; die Zuordnung von Zeitschriften über den Namen ist unvollkommen, wenn ein Datensatz keine ISSN hat.
- Deduplizierung erfolgt über die DOI, sonst über die ersten 90 Zeichen des Titels; Preprint/Publikations-Paare mit verschiedenen Titeln überleben als zwei Datensätze.
- Google Scholar ist und wird kein Backend (keine API; Scraping verletzt seine Bedingungen).

13. Tests

`python tests/test_librarian.py`

Offline, nur Standardbibliothek, keine Schlüssel: Backends laufen gegen aufgezeichnete API-Antworten, der Berichtsgenerator gegen synthetische Läufe und Forschungsverzeichnisse, und die Kommandozeile jedes Skripts wird von Ende zu Ende durchlaufen. Die Datei ist zugleich ein pytest-Modul (`pytest tests/`). Die CI führt pyflakes und die Suite auf Linux, Windows und macOS unter Python 3.9 und 3.13 aus.

14. Lizenz und Verhalten

Apache License 2.0. Das Werkzeug ist so gebaut, dass die Achtung der Nutzungsbedingungen jeder Datenbank der einfache Weg ist: nur dokumentierte APIs, eingehaltene Ratenlimits, eine Kontaktadresse in jeder Anfrage, kein Scraping, keine Umgehung von Bezahlschranken.